Industrial Security (Bachelor)

Solution Sheet – Section 04

Industrial Protocols

Note to graders. Each model answer is intentionally short. Award full credit for a different phrasing only if the student names the same wire-level mechanism or the same compensating control layer; "apply security" or "use a firewall" without specifics is not full credit.

Solution

Exercise 4.1 – Decode a Modbus TCP Request. MBAP header: Transaction ID `0x0001` (client correlation cookie), Protocol ID `0x0000` (always zero for Modbus), Length `0x0006` (number of bytes from the Unit ID onward), Unit ID `0x11` (decimal `17`, the slave or gateway target). Function code `0x03` = *Read Holding Registers*, which returns 16-bit values from the holding-register address space. Starting register `0x006B` (decimal `107`), quantity `0x0003`, so registers `107..109` are read. A successful response carries MBAP (7 B) + FC (1 B) + byte count (1 B) + 3×2 B of register data = 15 bytes total; the MBAP `Length` field on the response is `0x0009` (unit + FC + byte count + 6 data bytes). *Rubric: −1 for each field misnamed; −2 if Length is reported as 12 instead of 9.*

Solution

Exercise 4.2 – EtherNet/IP Scan Took the Line Down. (1) The CIP stack on an older ControlLogix CPU is single-threaded for explicit messaging; a burst of `ListIdentity`/`GetAttributesAll` requests can starve the scan task. In the PCAP, look for tens of explicit TCP/`44818` sessions opened in rapid succession from the scanner IP with no implicit UDP/`2222` traffic interrupted. (2) Connection-table exhaustion: each scan probe opens a CIP connection; ControlLogix has a finite Class 3 connection pool. Look for repeated `ForwardOpen`/`ForwardClose` pairs and, eventually, `ForwardOpen` responses with status `0x01` (*Connection Failure*). (3) A malformed or borderline-CIP request triggered an assertion in the firmware (this is the historical Project Basecamp / OT:ICEFALL pattern). Look for the last request before the silence; if the same byte pattern reproduces the freeze on a lab device, this is the cause. *Rubric: any three plausible OT-specific causes are acceptable; deduct if the student blames generic "DoS" without naming a CIP-side resource.*

Solution

Exercise 4.3 – Hardening a Mosquitto Broker.

```
listener 8883
cafile   /etc/mosquitto/ca.crt
certfile /etc/mosquitto/server.crt
keyfile  /etc/mosquitto/server.key
require_certificate true
allow_anonymous false
password_file /etc/mosquitto/passwd
```

Any four of the lines above count. `listener 8883` moves the broker off plaintext `1883/tcp`; the three TLS lines enable confidentiality and server authentication; `require_certificate true` forces *client* authentication (mutual TLS) so an attacker cannot just publish with a stolen password; `allow_anonymous false` together with `password_file` blocks the anonymous default that Shodan finds on hundreds of thousands of brokers. *Rubric: full credit only if the student justifies why each line matters; "it is more secure" is not a justification.*

Solution

Exercise 4.4 – "Use OPC UA, It Is Secure". The four configuration choices are: (1) *SecurityPolicy*: must be `Basic256Sha256` or `Aes256_Sha256_RsaPss`. The insecure default is `SecurityPolicy = None`, accepted by many configurators and visible on the wire as un-signed UA messages. (2) *MessageSecurityMode*: must be `SignAndEncrypt`. The insecure default is `None` or `Sign`-only. (3) *User authentication*: must use username/password or X.509 user tokens with a real identity. The insecure default is the `Anonymous` user token. (4) *Certificate trust list*: the server's trust list must contain only expected client certificates, and self-signed certificates must *not* be auto-accepted. The insecure default is "trust on first use" with auto-acceptance enabled for commissioning. *Rubric: accept "application-instance certificate lifecycle / GDS" as a substitute for (4); deduct if the student lists OPC UA features (e.g. "namespaces") instead of security choices.*

Solution

Exercise 4.5 – Why Modbus Has No Authentication. Modbus was designed in 1979 for short, point-to-point RS-485 loops inside one cabinet. The physical layer *was* the security boundary: if you were on the wire you were trusted, and CPU cycles on the 8-bit microcontrollers of the day were too scarce for any kind of crypto. Adding authentication later would have broken every installed device, so the specification was frozen. The modern compensating-control pattern is a layered one: (a) network segmentation that places Modbus devices in their own zone, (b) a stateful or deep-packet-inspection firewall at the conduit that allowlists function codes per source IP, (c) optionally Modbus Security (TCP/802 with TLS, specified in 2018) where both endpoints support it, and (d) monitoring on the segment for unexpected write function codes (`0x05/0x06/0x0F/0x10`). *Rubric: full credit needs both the "trust boundary was the wire" historical argument and at least two compensating layers.*

Solution

Exercise 4.6 – Reading a DNP3 Trace. Operationally, outstation `5` is pushing event data to master `1` without being polled. This is the canonical DNP3 unsolicited response pattern used so that, e.g., a relay can report a breaker trip immediately instead of waiting for the next master cycle. Six class-1 events in 50 ms is plausible for a real fault (a single bay can generate a burst of state changes). The anomaly worth checking is whether the burst correlates with anything in the SCADA event log: if the master reports no operator action and no upstream fault, the unsolicited burst could be a spoofed or replayed sequence (DNP3 without SAv5 has no authentication, so any host on the segment can forge a frame with source address `5`). Confirm by checking the application-layer sequence numbers for gaps or duplicates, by correlating with the IED's own SOE log, and by checking whether SAv5 challenge/response is configured on the link. *Rubric: accept any anomaly that ties to spoofing/replay or to event-storm overload; deduct if the student treats unsolicited responses themselves as the anomaly.*

Solution

Exercise 4.7 – Profinet vs. Modbus: Fail-Closed? Profinet fails closed first. Profinet RT runs a cyclic exchange with a configured update time (here 4 ms) and a watchdog factor (typically 3, giving a ∼12 ms timeout); an 800 ms outage is two orders of magnitude longer than the watchdog, so the controller drops the connection, the device enters its configured fail-safe substitute values, and the controller logs a diagnostic. Modbus TCP has no such watchdog: the HMI simply retries on the next 500 ms poll, the user sees a stale value for one cycle, and nothing in the protocol forces a safe state. Safety implication: a Profinet IO outage forces the process into its engineered safe substitute state (valves close, drives coast to stop), which is the desired behaviour on a safety-relevant interlock. A Modbus outage hides the problem from the operator,

who may act on stale data – which is why Modbus must never be the path for a safety function. *Rubric: must mention the Profinet watchdog and the absence of one in Modbus; deduct if the student claims Modbus also fails closed.*

Solution

Exercise 4.8 – Compensating for Unauthenticated GOOSE. (1) *Layer-2 segmentation*: put the GOOSE/SV bus on its own VLAN with no routed access. Cheap, no protocol change. Drawback: a compromised engineering laptop on the same VLAN bypasses it entirely. (2) *Bay-level monitoring*: deploy a passive IDS (e.g. a network tap plus an OT IDS) that learns the expected GOOSE multicast MAC addresses, state numbers, and sequence numbers, and alerts on duplicates or unexpected publishers. Moderate cost, no latency impact. Drawback: detection only – the breaker has already tripped by the time the alert arrives. (3) *IEC 62351-6 authentication*: enable HMAC-based message authentication on every IED and merging unit. Highest cost (firmware upgrades on every IED, key-management infrastructure, vendor support matrix) and the protection engineer will object that the additional processing pushes end-to-end latency past the 4 ms budget for transfer trip schemes. *Rubric: ordering may vary slightly, but the three controls must be distinct (network, monitoring, crypto) and the drawback must be operationally specific.*

Solution

Exercise 4.9 – Spot the Bug in a Modbus Client. (1) The client opens and closes a fresh TCP connection every 100 ms. On the PLC side this consumes a slot in the (small) Modbus TCP connection table – typical S7-1200 or ControlLogix devices allow only 4 to 16 concurrent connections, and each closed connection sits in `TIME_WAIT` on the PLC stack for tens of seconds. Under sustained polling the table fills, new connections are refused, and any other client (an HMI, the engineering workstation) stops being able to talk to the PLC. The operator sees the HMI go "no data". (2) The standard fix is to open the TCP connection once and reuse it for every transaction, multiplexing requests with the MBAP transaction ID. The Modbus Organization explicitly recommends this in the *Modbus/TCP Implementation Guide* precisely because PLC connection tables are small and `TIME_WAIT` on embedded stacks is expensive. *Rubric: accept "connection pooling" as the fix; deduct if the student blames bandwidth – the bug is connection-table exhaustion, not throughput.*

Solution

Exercise 4.10 – Three Security Models Side by Side.

	OPC UA `Basic256Sha256`	**MQTT over TLS**	**Profinet Security**
Auth	X.509 mutual (application instance) + user token	TLS server cert; optional mTLS; broker password	Certificate-based per IEC 62443, device identity
Integrity	RSA-SHA256 message signatures	TLS record MAC (AEAD)	HMAC over Profinet PDUs
Confidentiality	AES-256-CBC at message layer	TLS 1.2/1.3 AEAD (e.g. AES-GCM)	AES-GCM at PN frame layer
Key mgmt	PKI; optional GDS for cert push and CRL	Out-of-band PKI; broker-managed creds	Vendor PKI; engineering tool provisions certs

OPC UA is hardest to operate at scale because each application instance needs its own certificate, the trust lists must be kept consistent across servers and clients, and revocation must propagate before a stolen cert expires; without a GDS the work is manual per device. MQTT-TLS is the easiest because the broker is a single choke point and standard web-PKI tooling

applies. Profinet Security sits in between but is constrained by vendor support – only newer IO devices implement it. *Rubric: cell entries may use different but equivalent mechanism names; the operability paragraph must identify certificate lifecycle as the OPC UA pain point.*